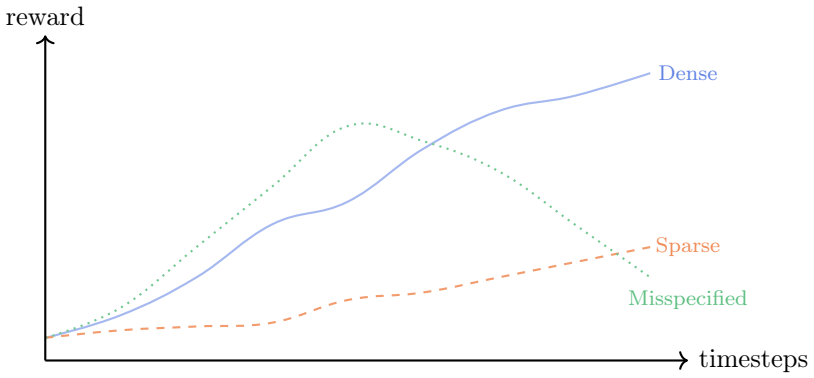
Train a Robot to Walk

Weeks 2 & 3: Reward Engineering & Hyperparameter Ablation



"The reward function is the only way you communicate your intent to the agent."

DC Study Guide

Robotics & ML Club · DC Training Program

Version 1.0 · 2025

Contents

1 What This Guide Covers **3**
1.1 Deliverables . . . 3
1.2 Files You'll Work With . . . 3

2 Reward Engineering — Theory **4**
2.1 Why Reward Functions Matter . . . 4
2.2 Sparse vs. Dense Rewards . . . 4
2.3 The Anatomy of a Dense Reward Function . . . 4
2.4 Reward Hacking: When the Agent Outsmarts You . . . 5
2.5 Reward Scaling: A Subtle But Critical Detail . . . 5

3 The Four Reward Functions **7**
3.1 Reward 1: Sparse Reward . . . 7
3.2 Reward 2: Dense Reward (Baseline) . . . 7
3.3 Reward 3: Velocity Only (No Energy Penalty) . . . 7
3.4 Reward 4: Heavy Energy Penalty . . . 7
3.5 Reward 5: Your Custom Reward (Creative Task) . . . 8

4 Implementing Reward Functions — Code Walkthrough **9**
4.1 Understanding the Interface . . . 9
4.2 Observation Vector Cheat Sheet . . . 9
4.3 Code Pattern: Dense Reward (Given) . . . 9
4.4 Code Pattern: Sparse Reward (TODO) . . . 9

5 Running Reward Experiments **11**
5.1 Experiment Script Overview . . . 11
5.2 How to Run . . . 11
5.3 What to Look For in TensorBoard . . . 11

6 Hyperparameter Ablation — Theory **12**
6.1 What Is an Ablation Study? . . . 12
6.2 Parameters You'll Ablate . . . 12
6.2.1 Learning Rate (α) . . . 12
6.2.2 Network Architecture . . . 13
6.2.3 Rollout Length (`n_steps`) . . . 13
6.2.4 Discount Factor (γ) . . . 13
6.2.5 Entropy Coefficient (`ent_coef`) . . . 13

7 Running Ablation Experiments **15**
7.1 The Ablation Script . . . 15
7.2 Expected Results . . . 15

8 Comparison and Visualization **16**
8.1 Generating Comparison Plots . . . 16
8.2 How to Read a Reward Curve . . . 16
8.3 Walk Success Rate . . . 16

9 Reward Engineering Principles — Summary **17**

10 Hyperparameter Intuitions — Summary **17**

11 Exercises and Checkpoints **18**
11.1 Week 2 Checkpoint (End of first week) 18
11.2 Week 3 Checkpoint (End of second week) 18
11.3 Self-Check Questions 18

12 Troubleshooting **19**

13 Further Reading **19**

What This Guide Covers

Welcome back! In Week 1 you set up the environment, trained a baseline PPO agent, and observed it learning to walk in TensorBoard. Now it's time to go deeper.

Over the next two weeks you will tackle the two most important applied skills in reinforcement learning:

1. **Reward Engineering** — designing, implementing, and comparing different reward functions to shape the behaviour you actually want.
2. **Hyperparameter Ablation** — systematically varying one parameter at a time and measuring the effect, the bread and butter of ML research.

Deliverables

By the end of Week 3, you should have:

- At least **4 reward functions** implemented, trained, and compared (including one you design yourself).
- At least **5 hyperparameter configurations** tested through structured ablation experiments.
- A **comparison plot** (reward curves on one graph) and a **results table**.
- Recorded **demo videos** of your best and worst agents.
- Short written analysis: what worked, what didn't, and *why*.

Files You'll Work With

File	Purpose
`reward_functions.py`	Implement all reward functions here. Has TODOs for you to fill.
`run_reward_experiments.py`	Trains agents with each reward function and logs results.
`run_ablation.py`	Runs hyperparameter ablation experiments. Has TODOs.
`compare_and_plot.py`	Loads logs, generates comparison plots and tables.

Reward Engineering — Theory

Why Reward Functions Matter

In supervised learning, the label tells the model the right answer directly. In RL, we don't have labels, we only have the reward signal. The reward function is our *only* channel of communication with the agent. Get it wrong, and the agent will happily optimize for something you never intended.

Key Insight

Designing a good reward function is the single most important applied RL skill. It's more impactful than choosing the right algorithm or tuning hyperparameters.

Sparse vs. Dense Rewards

Definition 2.1 (Sparse Reward)**.** A reward function that returns a non-zero signal only at rare events (e.g., reaching a goal, falling).

Definition 2.2 (Dense Reward)**.** A reward function that provides a meaningful signal at **every** timestep, guiding the agent continuously.

Analogy. Imagine learning to play basketball. A sparse reward is like only being told "you won" or "you lost" at the end of the game, you get no feedback on individual shots, passes, or defensive plays. A dense reward is like having a coach who gives real-time commentary: "nice follow-through", "move your feet", "pass earlier". Which do you think helps you improve faster?

Mathematically, consider a 1000-step walking episode:

- **Sparse**: 998 timesteps give $r_t = 0$. The agent receives almost no gradient signal. Learning requires millions of steps of random exploration to stumble onto the good transitions.
- **Dense**: Every timestep gives $r_t = f(v_x, h, a, \ldots)$. The agent can immediately tell whether its last action moved it in the right direction.

The Anatomy of a Dense Reward Function

A well-designed dense reward for locomotion typically has this structure:

$$\boxed{r_t = \underbrace{w_1 \cdot \text{progress}}_{\text{what you want}} + \underbrace{w_2 \cdot \text{alive}}_{\text{survival incentive}} - \underbrace{w_3 \cdot \text{penalties}}_{\text{what you don't want}} + \underbrace{w_4 \cdot \text{style}}_{\text{nice to have}}} \tag{1}$$

Each component serves a distinct purpose:

Component	Role
Progress	The primary objective. For walking: forward velocity.
Alive bonus	Constant reward for not falling. Prevents the agent from "giving up" by falling to end a bad episode quickly.
Energy penalty	Discourages wasteful, jerky, high-torque movements. Produces smoother gaits.
Lateral penalty	Prevents sideways drift. Keeps the robot on a straight line.
Height/posture bonus	Encourages upright walking rather than crouching.

Reward Hacking: When the Agent Outsmarts You

Common Mistake

Reward hacking occurs when the agent finds a way to get high reward without doing what you actually intended. The agent is *always* right, if it finds a loophole, the problem is your reward function, not the agent.

Classic examples:

- A hopping robot learns to bounce in place because "not falling" gives a bigger cumulative reward than the risky act of walking forward.
- An agent rewarded for forward velocity learns to fling itself forward in one massive lunge, maximizing instantaneous speed before crashing.
- A robot penalized for energy use learns to stand perfectly still, zero energy, zero penalty, infinite alive bonus.

Each of these is a *valid* optimum of the given reward function. The fix is always to redesign the reward, never to "tell the agent to stop cheating."

Reward Scaling: A Subtle But Critical Detail

Warning

If one reward component gives ~ 2.0 per step and another gives ~ 0.001, the small component is effectively invisible to the optimizer. Always check the **typical magnitude** of each component during early training.

For our walker with the default dense reward:

Component	Typical value	Coefficient
Forward velocity	0–5.0	$c_1 = 1.0$
Alive bonus	1.0 (constant)	$c_2 = 1.0$
Energy penalty	0.001–0.01	$c_3 = 0.001$
Lateral penalty	0–1.0	$c_4 = 0.5$
Height bonus	0–0.1	$c_5 = 0.1$

The forward velocity and alive bonus dominate, which is by design, walking forward and staying alive are the primary objectives. The energy penalty is intentionally small: just enough to smooth the gait without blocking forward progress.

The Four Reward Functions

You will implement all four of these. The first two are partially given; the rest have TODOs for you to complete.

Reward 1: Sparse Reward

$$r_t = \begin{cases} -1.0 & \text{if robot fell} \\ +1.0 & \text{if } \Delta x > 0.01 \text{ m} \\ 0.0 & \text{otherwise} \end{cases} \tag{2}$$

What to expect: The agent gets almost no useful feedback. In a 22-dimensional continuous state space, the probability of randomly discovering good walking motions is negligible. You'll see near-zero reward curves for hundreds of thousands of steps. If walking emerges at all, it will be after 2–5 million steps.

Why we test it: To demonstrate that sparse rewards are a terrible idea for continuous control tasks. This is the control experiment.

Reward 2: Dense Reward (Baseline)

$$r_t = \underbrace{1.0 \cdot \text{clip}(v_x,\, 0,\, 5)}_{\text{forward vel.}} + \underbrace{1.0}_{\text{alive}} - \underbrace{0.001 \cdot \|a\|_2^2}_{\text{energy}} - \underbrace{0.5 \cdot |v_y|}_{\text{lateral}} + \underbrace{0.1 \cdot \max(0,\, 1 - |h - 1.2|)}_{\text{height}} \tag{3}$$

What to expect: Learning begins immediately. By 200k–400k steps, you should see the robot walking with a recognizable bipedal gait. Reward typically reaches 1800–2500 by 500k steps.

Why this is the baseline: Every component is carefully balanced. The velocity clip at 5 m/s prevents pathological speed-chasing. The alive bonus prevents premature termination. The energy penalty smooths the gait without killing forward progress.

Reward 3: Velocity Only (No Energy Penalty)

Same as the dense reward but with $c_3 = 0$ (no energy penalty).

$$r_t = 1.0 \cdot \text{clip}(v_x,\, 0,\, 5) + 1.0 - 0.5 \cdot |v_y| + 0.1 \cdot \max(0,\, 1 - |h - 1.2|) \tag{4}$$

What to expect: The agent walks *faster* but with ugly, jerky, aggressive movements. Joints slam between extremes because there's no cost for high-torque actions. This experiment isolates exactly what the energy penalty does.

Observation Task

After training both Dense and Velocity-Only, record videos of each. Watch them side by side. In 3–4 sentences, describe: what is visually different about the gaits? Can you connect the difference to the missing energy penalty term?

Reward 4: Heavy Energy Penalty

Dense reward but with $c_3 = 0.01$ (10× the default).

$$r_t = 1.0 \cdot \text{clip}(v_x,\, 0,\, 5) + 1.0 - \mathbf{0.01} \cdot \|a\|_2^2 - 0.5 \cdot |v_y| + 0.1 \cdot \max(0,\, 1 - |h - 1.2|) \tag{5}$$

What to expect: One of two outcomes (seed-dependent):

1. **Conservative walker**: Very slow, minimal-effort walking. Energy-efficient but sluggish.
2. **Policy collapse**: The energy penalty outweighs the velocity reward, so standing still becomes the global optimum. The agent learns to do *nothing*.

This is a textbook case of **reward misspecification**, the weights don't express what you want.

Reward 5: Your Custom Reward (Creative Task)

Code TODO — Your Turn!

Design your own reward function! Some ideas:

- Add a **smoothness penalty**: penalize the *change* in action between consecutive steps, i.e., $-w \cdot \|a_t - a_{t-1}\|^2$.
- Add a **symmetry bonus**: reward alternating foot contacts (proper bipedal gait) with something like $+w \cdot |c_L - c_R|$.
- Add a **velocity target**: instead of clipping, reward proximity to a target speed, e.g., $-w \cdot (v_x - v^*)^2$ for some $v^* = 2.0$.
- Combine any of the above with the dense reward.

You must explain: (a) what behaviour you're trying to encourage, (b) the mathematical formula, and (c) what you observed during training.

Implementing Reward Functions — Code Walkthrough

Open `reward_functions.py`. Let's walk through the structure.

Understanding the Interface

Every reward function receives three arguments and returns a float:

```
def my_reward(obs: np.ndarray, action: np.ndarray,
              info: dict) -> float:
    """
    Args:
        obs:    shape (22,) -- the observation vector
        action: shape (6,)  -- the torques applied
        info:   dict with keys like 'fell', 'dx', etc.
    Returns:
        float: scalar reward for this timestep
    """
```

Observation Vector Cheat Sheet

You'll need these indices constantly while writing reward functions:

Index	Meaning	Index	Meaning
`obs[0]`	Torso height (m)	`obs[14]`	Forward velocity v_x
`obs[1]`	Torso pitch (rad)	`obs[15]`	Lateral velocity v_y
`obs[2:8]`	Joint angles	`obs[16]`	Vertical velocity v_z
`obs[8:14]`	Joint velocities	`obs[17]`	Pitch rate
		`obs[18]`	Left foot contact
		`obs[19]`	Right foot contact

Code Pattern: Dense Reward (Given)

This is implemented for you as a reference. Study it carefully, you'll follow the same pattern for the others.

```
def dense_reward(obs, action, info):
    forward = 1.0 * np.clip(obs[14], 0.0, 5.0)
    alive   = 1.0
    energy  = 0.001 * np.sum(np.square(action))
    lateral = 0.5 * abs(obs[15])
    height  = 0.1 * max(0.0, 1.0 - abs(obs[0] - 1.2))

    return forward + alive - energy - lateral + height
```

Code Pattern: Sparse Reward (TODO)

In `reward_functions.py`, you'll find this skeleton:

```
def sparse_reward(obs, action, info):
    # TODO: Implement sparse reward
```

```
    # Return -1.0 if the robot fell (info['fell'] is True)
    # Return +1.0 if forward displacement > 0.01
    # Return  0.0 otherwise
    #
    # Hint: info['dx'] gives forward displacement
    #        this step, info['fell'] is a boolean
    pass  # <-- replace this
```

Key Insight

After implementing each reward function, **test it manually** before training. Print the reward for a few random observations and actions. Does the output make sense? Are the magnitudes reasonable?

Running Reward Experiments

Experiment Script Overview

The script `run_reward_experiments.py` automates training for each reward function. It:

1. Loops through a dictionary of reward configurations.
2. Creates a separate environment wrapper for each reward.
3. Trains a PPO agent for 500k timesteps.
4. Saves the model and TensorBoard logs under `logs/reward_<name>/`.

How to Run

```
# Train all reward experiments (takes ~1-2 hours total)
python run_reward_experiments.py

# Train just one specific reward
python run_reward_experiments.py --reward dense

# Monitor in TensorBoard (open a second terminal)
tensorboard --logdir logs/
```

What to Look For in TensorBoard

Metric	What to Check
`rollout/ep_rew_mean`	Main reward curve. Compare shapes across reward functions: does it go up? Plateau? Collapse?
`rollout/ep_len_mean`	Episode length. If it's stuck at a low number, the agent keeps falling early.
`train/entropy_loss`	Is the agent still exploring? If this drops to near zero too fast, it's stuck.
`train/approx_kl`	Should stay below ∼0.02. Spikes here mean the policy changed too much in one update.

TensorBoard Comparison Task

Open TensorBoard and display all four reward curves on the same plot (TensorBoard does this automatically when they're in sibling directories under `logs/`). Take a screenshot. In 2–3 sentences, rank the reward functions from best to worst and explain the ranking.

Hyperparameter Ablation — Theory

What Is an Ablation Study?

An ablation study changes **exactly one variable** at a time while keeping everything else fixed, then measures the effect. It's the controlled experiment of machine learning research.

Key Insight

The golden rule of ablation: **change one thing, measure one thing**. If you change the learning rate *and* the network size simultaneously, you can't attribute the result to either one.

Your baseline configuration (the "control") is the default PPO setup from Week 1:

Parameter	Baseline Value	What It Controls
`learning_rate`	3×10^{-4}	Gradient step size
`n_steps`	2048	Rollout length before each update
`batch_size`	64	Mini-batch size within each update
`n_epochs`	10	Number of passes over collected data
`gamma`	0.99	Discount factor
`gae_lambda`	0.95	GAE bias–variance trade-off
`clip_range`	0.2	PPO clipping parameter ϵ
`ent_coef`	0.0	Entropy bonus weight
`policy_kwargs`	`{net_arch: [256, 256]}`	Hidden layer sizes

Parameters You'll Ablate

We'll ablate five parameters (one at a time). For each, you test 2–3 values against the baseline.

6.2.1 Learning Rate (α)

The learning rate controls how big each gradient step is:

$$\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$$

- $\alpha = 3 \times 10^{-4}$ (baseline): Stable for locomotion. Well-tested default.
- $\alpha = 1 \times 10^{-3}$ (high): Faster early progress but the reward curve may oscillate wildly or even *diverge*. Watch for sudden drops.
- $\alpha = 1 \times 10^{-5}$ (low): Extremely stable but painfully slow. The agent may barely walk after 500k steps.

Warning

If your reward curve suddenly crashes to zero or becomes very noisy after initially rising, the learning rate is almost always too high. This is the single most common training failure in RL.

6.2.2 Network Architecture

The policy and value networks are MLPs (multi-layer perceptrons). We vary the hidden layer sizes:

- `[64, 64]` (small): Only ∼8,000 parameters. May not have enough capacity to represent complex walking dynamics. Expect a simpler, possibly unstable gait.
- `[256, 256]` (baseline): ∼120,000 parameters. Good balance of capacity and training speed.
- `[512, 512]` (large): More parameters but **not necessarily better**. In RL (unlike supervised learning), more capacity doesn't always help because the training signal is noisy and non-stationary. Expect similar or slightly worse results with slower wall-clock time.

6.2.3 Rollout Length (`n_steps`)

This controls how many timesteps of experience are collected before each PPO update.

$$\text{Data per update} = \texttt{n_steps} \times \texttt{n_envs}$$

With `n_envs` = 4:

- `n_steps` = 512 (small): 2,048 transitions per update. Advantages are estimated from short trajectory segments, leading to high variance. Updates are fast but noisy.
- `n_steps` = 2048 (baseline): 8,192 transitions. Well-studied default with enough diversity for stable gradients.
- `n_steps` = 4096 (large): 16,384 transitions. Smoother reward curves but each update takes longer in wall-clock time.

6.2.4 Discount Factor (γ)

γ controls how much the agent cares about future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

A reward 100 steps in the future is worth γ^{100} of an immediate reward:

- $\gamma = 0.95$: $0.95^{100} \approx 0.006$. The agent is very myopic, only the next ∼20 steps matter. May learn to stand still rather than invest in learning to walk.
- $\gamma = 0.99$ (baseline): $0.99^{100} \approx 0.37$. Good balance for locomotion.
- $\gamma = 0.999$: $0.999^{100} \approx 0.90$. Values future rewards almost as much as immediate ones. Can cause instability because small value estimation errors propagate far.

6.2.5 Entropy Coefficient (`ent_coef`)

The entropy bonus encourages exploration by rewarding stochastic (uncertain) policies:

$$\mathcal{L}_{\text{total}} = -\mathcal{L}^{\text{CLIP}} + c_1 \mathcal{L}^{\text{VF}} - c_2 \underbrace{H[\pi_\theta(\cdot|s)]}_{\text{entropy bonus}}$$

- $c_2 = 0.0$ (baseline): No explicit exploration bonus. The agent relies on the initial random policy for exploration.

- $c_2 = 0.01$: Mild exploration bonus. Can help escape local optima early in training but may slow convergence later.
- $c_2 = 0.1$: Strong exploration bonus. The agent keeps acting randomly for longer. Usually *hurts* performance because the agent never commits to a good policy.

Running Ablation Experiments

The Ablation Script

Open `run_ablation.py`. The structure mirrors the reward experiments but now varies hyperparameters instead of reward functions.

```
# In run_ablation.py you'll define experiments like:
ABLATION_CONFIGS = {
    "baseline": {},  # all defaults
    "lr_high":  {"learning_rate": 1e-3},
    "lr_low":   {"learning_rate": 1e-5},
    # TODO: Add your network architecture ablations
    # TODO: Add your n_steps ablations
    # TODO: Add gamma and ent_coef ablations
}
```

Each experiment trains for 500k steps and logs to `logs/ablation_<name>/`.

Expected Results

Use this table as a rough guide. Your actual numbers will vary with the random seed, but the *relative ranking* should be consistent.

Config	Reward Range	Walk %	Notes
Baseline	1800–2500	80–100%	Reference point
LR $= 10^{-3}$	1500–2500	60–90%	May show instability spikes
LR $= 10^{-5}$	500–1500	20–60%	Clearly slower convergence
Net 64×64	1200–2000	50–80%	Simpler gait, less stable
`n_steps` = 4096	1800–2800	80–100%	Smoother curves
$\gamma = 0.95$	800–1800	40–70%	Short-sighted behaviour
`ent_coef` = 0.01	1600–2400	70–90%	More exploration early

Ablation Analysis Task

After running all ablations, fill in the results table in your code (the script will generate a CSV). Then answer: **Which single parameter had the biggest impact on performance? Why do you think that is?** Write 3–5 sentences.

Comparison and Visualization

Generating Comparison Plots

The `compare_and_plot.py` script reads TensorBoard event files and generates publication-quality plots.

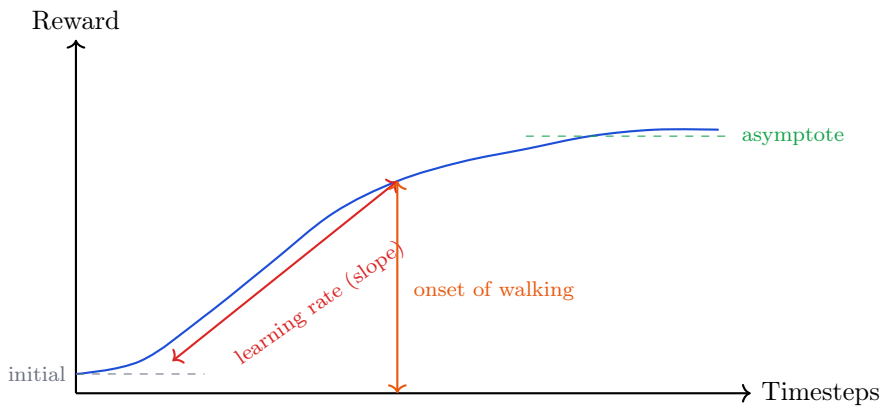
```
# Generate all comparison plots
python compare_and_plot.py --logdir logs/ --output plots/
```

This produces:

- `reward_comparison.png` — All reward function curves overlaid.
- `ablation_comparison.png` — All ablation curves overlaid.
- `results_table.csv` — Summary statistics for your report.

How to Read a Reward Curve

A reward curve has three important features:



1. **Slope** (learning speed): How quickly does the curve rise? A steep initial slope means the agent found useful behaviour early.
2. **Asymptote** (final performance): Where does the curve plateau? This is the quality of the final policy.
3. **Stability**: Is the curve smooth or noisy? High variance (jagged curves) suggests unstable training, possibly too high a learning rate or too short a rollout.

Walk Success Rate

We define a "successful walk" as the agent moving at least **3 meters forward** in a single episode without falling. Your evaluation script computes this over 20 episodes.

```
# In your evaluation code:
success = sum(1 for ep in episodes
              if ep['distance'] >= 3.0
              and not ep['fell']) / len(episodes)
```

Reward Engineering Principles — Summary

These are the five principles you should internalize from Weeks 2–3. They apply to **any** RL project, not just walking robots.

1. **Dense > Sparse for most tasks.** Always ask yourself: "Can I provide feedback at every timestep, not just at the end?"
2. **Always include a survival bonus.** Without it, falling quickly is often the rational choice for a struggling agent. The alive bonus makes continued existence worthwhile.
3. **Penalize what you don't want.** Don't just reward forward velocity, also penalize energy waste, sideways drift, and instability. Every unpenalized behaviour is a potential loophole.
4. **Balance your component scales.** Each reward term should contribute meaningfully during early training. If one term is 1000× larger than another, the small one is invisible.
5. **Test by hand first.** Before launching a training run, compute the reward for a few hand-crafted states. Does a "standing still" state give less reward than a "walking forward" state? Does a "fallen" state give negative reward? If the math doesn't check out, don't train.

Hyperparameter Intuitions — Summary

Parameter	Too High	Too Low
Learning rate	Reward oscillates, may crash to zero	Convergence is painfully slow
Network size	Slower updates, no real benefit	Underfits complex gaits
`n_steps`	Slow wall-clock time	High-variance, noisy learning
γ	Unstable credit assignment	Myopic, short-sighted agent
`ent_coef`	Never converges (too random)	Gets stuck in local optima
`clip_range`	Large, destabilizing updates	Too conservative, no learning

Exercises and Checkpoints

Week 2 Checkpoint (End of first week)

Reward Engineering Checkpoint

By the end of the first week in this block, you should have:

- ☐ Implemented all 4 given reward functions in `reward_functions.py`.
- ☐ Trained each for 500k steps.
- ☐ Taken a TensorBoard screenshot showing all 4 curves overlaid.
- ☐ Written 3–5 sentences comparing the reward functions.
- ☐ Recorded a video of the dense-reward agent walking.

Week 3 Checkpoint (End of second week)

Ablation Checkpoint

By the end of the second week, you should have:

- ☐ Completed all TODOs in `run_ablation.py`.
- ☐ Trained at least 5 ablation configurations (baseline + 4 variations).
- ☐ Filled in the results table (auto-generated CSV).
- ☐ Generated comparison plots.
- ☐ Written a short analysis: which parameter mattered most and why.
- ☐ Designed and trained your custom reward function (Reward 5).

Self-Check Questions

Use these to test your own understanding. If you can answer all of them confidently, you're on track.

1. Why do we need an alive bonus in the reward function?
2. The TensorBoard `value_loss` is going up during early training. Is that bad? Why or why not?
3. Why do we use 4 parallel environments instead of 1?
4. Your heavy-energy-penalty agent learned to stand still. Explain why, in terms of the reward function.
5. What would happen to the return G_t if $\gamma = 0$?
6. You changed the learning rate *and* the network size at the same time and got better results. Can you conclude that both changes helped? Why or why not?
7. How does the PPO clip parameter ϵ relate to training stability?

Troubleshooting

Symptom	Likely Cause	Fix
Reward collapses after 200k steps	Learning rate too high, or a reward spike broke the value function	Reduce LR to `1e-4`; check for NaN in rewards
Robot falls immediately every episode	Reward function bug (wrong sign, missing alive bonus)	Print rewards for first 10 steps; verify signs
Robot walks sideways	Missing lateral penalty	Add $-c_4\|v_y\|$ to reward
Training extremely slow	Too many parallel envs or rendering is on	Set `n_envs = 2`; ensure `render_mode` is `DIRECT`
TensorBoard not updating	Log directory mismatch	Check `-logdir` matches the path in your script
`ValueError` when loading a model	`VecNormalize` stats not saved	Save `vecnorm.pkl` alongside the model

Further Reading

If you want to go deeper after this project:

- **PPO Paper**: Schulman et al., "Proximal Policy Optimization Algorithms" (2017). `arxiv.org/abs/1707.0`
- **GAE Paper**: Schulman et al., "High-Dimensional Continuous Control Using Generalized Advantage Estimation" (2015). `arxiv.org/abs/1506.02438`
- **Reward Hacking**: Krakovna et al., "Specification gaming: the flip side of AI ingenuity" (DeepMind Blog, 2020).
- **OpenAI Spinning Up**: Hands-on RL codebase with excellent explanations. `spinningup.openai.com`
- **Stable-Baselines3 Docs**: `stable-baselines3.readthedocs.io`